

thinkdigit.com, and its text (which will become its inner HTML) to Think Digit Homepage. Finally, set the Target to `_blank` to render the content in a new unnamed window without frames.

Setting up Users and Roles

You'd prefer your personal web site to make sure users "log in" before viewing your personal content. This allows the site to restrict access to random people, and also allows the user to personalise the site to their individual needs.

Not long ago, creating the framework of logging in was a tedious and time consuming job: you had to create a database to track users, create the ability to maintain secure passwords by hard coding, ensure authorisation on each page, assign users to "roles" (such as guest, member, owner and wizard), and so on. You also had to write code for all the controls that allow the user to log in, to recover passwords, to change passwords, and so forth.

Click the Security tab. You'll see a window that gives you links to set the authentication type, define authorisa-



Set parameters for your web site, without bothering about code

FORM AUTHENTICATION

Enable forms authentication and denying anonymous users in the web.config:

```
<configuration>
<system.web>
...
<authentication mode="Forms">
<forms loginUrl="~/Login.aspx" />
</authentication>
<authorization>
<deny users="?" />
<allow users="*" />
</authorization>
</system.web>
```

Now, users will be redirected to a login page named Login.aspx that you need to create. When the user clicks on the Login button, the page checks whether the user has typed in the password 'zxcvqwerty' and then uses the `RedirectFromLoginPage()` method to log the user in. Here's the complete page code:

```
public partial class Login : System.Web.UI.Page
{
protected void cmdLogin_Click(Object sender, EventArgs e)
{
if (txtPassword.Text.ToLower() == "zxcvqwerty")
{
FormsAuthentication.RedirectFromLoginPage(txtName.Text, false);
}
else
{
lblStatus.Text = "Try again.";
}
}
```

tion rules (using the access rules section), and enable role-based security for your web site.

To set an application to use forms authentication, click on Select Authentication Type.

Choose From the Internet. Click on Done. The appropriate authorisation tag will be created in the `web.config` file automatically.

Next, define the authorisation rules for different types of users. To do so, click on the Create Access Rules link. To manage multiple rules, you'll need to click the Manage Access Rules link.

You can change the order of rules and hence the priority. If you have a large number of rules to create, it's easier to edit the `web.config` file. Once you've specified the authentication mode and the authorisation rules, you need to build the actual login page, which is an ordinary ASPX page that requests information from the user and decides whether the user should be authenticated or not.

Putting together the pieces

You've run all the pages from within Visual Studio 2010, and the web server you've been using is the ASP.NET Development Server that's built into Visual Studio. This set-up is good for testing. However, you can't run a public web site this way. For starters, it's unlikely that your computer is set up as a web server.

The ASP.NET Development Server that you've been using can't be used to serve a public web site as it accepts requests only from the local computer. You have a couple of options to get your site live that depend on how much of the site administration you want to take on. The easiest and most common option is to rent space from a hosting company and link it to your domain. Another option is to host the web site on a computer you maintain yourself. This is less common, but certainly possible.

There you have a functional web site with user registration, your personal information, photos, session state, and a consistent look and feel, all coded by you. You can now go out and create



Post your comments at
<http://thinkdigit.com/d/dec09/>



sites that you didn't dream were possible – before you read this, that is. There are plenty of more features that you could add to this site, such as more complex validation, user profiles, data handlers, image gallery and LINQ. The example here provides a good start for additional exploration in web development using Visual Studio. **1**